

Spencer Collins

Intro To C Final Report

8/6/2026

I would say I am pretty confident I “know” C. I feel prepared for GIOS in the Fall of 2026. I would be comfortable taking a job in C.

I learned about the call stack and stack frames at a granular level. It makes sense now how variables are locally scoped in a function and how global variable values can be recalled after being defined.

I think the assignment most interesting was the stackman interpreter. That one helped me with the overall interpreter project.

Most difficult was time management.

I would remove the assignments or break the interpreter phases down into weekly assignments instead.

I would give myself a grade of 2.0. I got most of phase 3 implemented and failed 2 tests on the test-basic.psi. I did not implement the proper error handling system and only relied on custom user error messages. There are memory leaks in my program as I used Valgrind to find them. I did not have time to implement the put-file and get-file. I also hardcoded in all the protected symbols for the global environment. I got protected symbol error when trying to run the other files. I would not use this language in production since it lacks many of the standard libraries found in Python or JavaScript. It is also not finished. I will finish the specification after this semester is over.

```
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\psi_phase1> ./psi "test-hardest.psi"
$error{error: unable to find binding for symbol...}
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\psi_phase1>

PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\psi_phase1> gcc -Wall -Wextra -std=c99 src/psi.c src/lexer.c src/parser.c src/eval.c src/globals.c -g -o psi
PS C:\Users\spenc\OneDrive\Desktop\GA Tech Online CS Master's Degree\CS8001-OIC_Intro_to_C_Programming\psi_phase1> ./psi test-basic.psi
"Test failed: " "try-B"
"Test failed: " "read-F"
Invalid token error
292 tests passed.
2 tests failed.
```

```

==6226==
==6226== HEAP SUMMARY:
==6226==   in use at exit: 764 bytes in 21 blocks
==6226==   total heap usage: 219,290 allocs, 219,269 frees, 10,859,190 bytes allocated
==6226==
==6226== 116 (24 direct, 92 indirect) bytes in 1 blocks are definitely lost in loss record 5 of 9
==6226==    at 0x4859858: malloc (vg_replace_malloc.c:447)
==6226==    by 0x4002AFD: pval_error (parser.c:102)
==6226==    by 0x4007EBD: eval_tree (eval.c:1816)
==6226==    by 0x400768E: eval_tree (eval.c:1646)
==6226==    by 0x400821B: eval_tree (eval.c:1892)
==6226==    by 0x4008E9B: eval_tree (eval.c:2179)
==6226==    by 0x4008C58: eval_tree (eval.c:2115)
==6226==    by 0x4001C93: main (psi.c:65)
==6226==
==6226== 264 (48 direct, 216 indirect) bytes in 2 blocks are definitely lost in loss record 8 of 9
==6226==    at 0x4859858: malloc (vg_replace_malloc.c:447)
==6226==    by 0x4002AFD: pval_error (parser.c:102)
==6226==    by 0x4007E12: eval_tree (eval.c:1798)
==6226==    by 0x400768E: eval_tree (eval.c:1646)
==6226==    by 0x400821B: eval_tree (eval.c:1892)
==6226==    by 0x4008E9B: eval_tree (eval.c:2179)
==6226==    by 0x4008C58: eval_tree (eval.c:2115)
==6226==    by 0x4001C93: main (psi.c:65)
==6226==
==6226== 384 (96 direct, 288 indirect) bytes in 4 blocks are definitely lost in loss record 9 of 9
==6226==    at 0x4859858: malloc (vg_replace_malloc.c:447)
==6226==    by 0x4002AFD: pval_error (parser.c:102)
==6226==    by 0x4007D34: eval_tree (eval.c:1780)
==6226==    by 0x400768E: eval_tree (eval.c:1646)
==6226==    by 0x400821B: eval_tree (eval.c:1892)
==6226==    by 0x4008E9B: eval_tree (eval.c:2179)
==6226==    by 0x4008C58: eval_tree (eval.c:2115)
==6226==    by 0x4001C93: main (psi.c:65)
==6226==
==6226== LEAK SUMMARY:
==6226==   definitely lost: 168 bytes in 7 blocks
==6226==   indirectly lost: 596 bytes in 14 blocks
==6226==   possibly lost: 0 bytes in 0 blocks
==6226==   still reachable: 0 bytes in 0 blocks
==6226==   suppressed: 0 bytes in 0 blocks
==6226==
==6226== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)

```


Eval.h

```
#include "../include/lexer.h"
#include "../include/parser.h"

//data structures
typedef struct binding binding;
struct binding {
    char* name;
    pval* value;
    struct binding* next;
};

typedef struct environment environment;

struct environment {
    binding *bindings; //bindings for this local environment
    struct environment *parent; // search parent environment if search doesn't
    find anything in this local environment
};

bool check_protected_symbols(char* name);

typedef struct closure closure;
```

```
struct closure {
    bool is_macro;

    pval* fn_name; //a pval_symbol

    pval* arglist;

    pval* body;

    environment* env;
};

// global/local environment functions

environment* create_env();

void add_env_parent(environment* env, environment* parent);

binding* create_binding(char* name, pval* value);

void env_add_binding(environment* env, binding* func);

pval* lookup_env_binding(environment* env, char* binding_name);

void delete_env(environment* env);

void delete_env_chain(environment* env);
```

```
//helper and evaluate functions

pval* pval_closure(bool is_macro, pval* fn_name, pval* arglist, pval* body,
environment* env);

pval* pval_copy(pval* pv);

pval* check_type(list_child* x);

pval* eval_tree(environment* env, pval* x);

// built in functions

pval* add(pval* list);

pval* subtract(pval* list);

pval* multiply(pval* list);

pval* divide(pval* list);

pval* equals(pval* list);
```

```
pval* gt(pval* list);
```

```
pval* lt(pval* list);
```

```
pval* gte(pval* list);
```

```
pval* lte(pval* list);
```

```
pval* not_eq(pval* list);
```

```
pval* modulo(pval* list);
```

```
pval* head(pval* list);
```

```
pval* tail(pval* list);
```

```
pval* cons(pval* list);
```

```
pval* not(pval* list);
```

```
pval* type(pval* x);
```

```
pval* exit_prg(pval* list);
```

```
pval* ord(pval* list);
```

```
pval* str_to_chr(pval* list);
```

```
pval* input(pval* str);
```

```
pval* output(pval* list);
```

```
pval* create_cell(pval* list);
```

```
pval* deref_cell(pval* list);
```

```
pval* write_cell(pval* list);
```

```
pval* str(pval* list);
```

```
pval* print(pval* list);

pval* error(pval* list);

Node* parse_help(pval* list);

pval* read(pval* list);

pval* make_list(pval* list);

pval* load(pval* list);

pval* get_file(pval* list);

pval* put_file(pval* list);
```

Globals.h

```
#ifndef GLOBALS
#define GLOBALS

extern bool terminate; //global lag for terminating program
```

```
#endif
```

Lexer.h

```
#ifndef LEXER
#define LEXER

typedef struct Node {
    char *token;
    struct Node *next;
} Node;

void printList(Node *node);

void deleteList(Node *head);

char *createToken(char command);

Node *createNode(char *token);

void append(Node **head, char *token);
```

```
Node *lex(Node *head, char *command);
```

```
#endif
```

Parser.h

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
#include "../include/lexer.h"
```

```
#ifndef PARSER
```

```
#define PARSER
```

```
typedef enum {
```

```
    PVAL_NUMBER,
```

```
    PVAL_BOOL,
```

```
    PVAL_SYMBOL,
```

```
    PVAL_LIST,
```

```
    PVAL_FUNCTION,
```

```
    PVAL_ERROR,
```

```
    PVAL_STRING,
```

```
    PVAL_CLOSURE,
```

```
    PVAL_CELL  
}  
pval_tag;  
  
typedef struct pval pval;  
  
typedef pval* (*pval_function)(pval*);  
  
typedef struct pv_func pv_func;  
  
struct pv_func {  
    char* func_name;  
    pval_function pval_fn_def;  
};  
  
typedef struct pv_str pv_str;  
  
struct pv_str {  
    int str_len;  
    char *str;  
};  
  
struct pval {  
    pval_tag pval_type;
```

```
int64_t reference_count;

union {

    struct closure *pval_closure;

    int64_t pval_number;

    bool pval_bool;

    struct pv_str *pval_string;

    char *pval_symbol;

    struct list *pval_list;

    struct pv_func *pval_func;

    struct pval *pval_error;

    struct pval *pval_cell;

};

};
```

```
typedef struct list_child {

    pval *atom;


    struct list_child *next;

} list_child;
```

```
typedef struct list {

    int list_count;


    list_child *atoms_head;
```

```
} list;

typedef struct function {
    int r;
} function;

pval* pval_func(pval_function pval_fn, char* func_name);

pval* pval_number(int64_t n);

pval* pval_bool(bool b);

pval* pval_error(pval* x);

pval* pval_cell(pval* x);

pval* pval_symbol(char *symbol);

pval* pval_string(char *str, int str_len);

pval* empty_list();
```

```
void pval_add(pval* list, pval* elem);

void pval_print(pval* pv);

void pval_delete(pval* pv);

pval* parse_atom(Node** current);

pval* parse_list(Node** current);

pval* parse_expression(Node** current);

#endif
```

Eval.c

```
#include <string.h>
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <inttypes.h>
```

```
#include <errno.h>
```

```
#include "../include/lexer.h"
```

```
#include "../include/parser.h"
```

```
#include "../include/eval.h"
```

```
#include "../include/globals.h"
```

```
pval* pval_closure(bool is_macro, pval* fn_name, pval* arglist, pval* body, environment* env)
```

```
{
```

```
    pval* pv_closure = (pval *)malloc(sizeof(pval));
```

```
    pv_closure->pval_type = PVAL_CLOSURE;
```

```
    pv_closure->reference_count = 1;
```

```
    //allocate memory for a new pval closure
```

```
    closure* pv_clos = (closure *)malloc(sizeof(closure));
```

```
    //copy fn_name if not NULL, otherwise NULL
```

```
    pv_clos->fn_name = NULL;
```

```
    if (fn_name != NULL) { pv_clos->fn_name = pval_copy(fn_name); }
```

```
    //assign the is_macro flag here
```

```
    pv_clos->is_macro = is_macro;
```

```
    //assign user defined function arguments to the closure,
```

```
    //need to pval_copy since the main() frees the evaled_tree which this gets called from
```

```
    pv_clos->arglist = pval_copy(arglist);
```

```
    //assign user defined function body to the closure,
```

```
//need to pval_copy since the main() frees the evald_tree which this gets called from
```

```
pv_clos->body = pval_copy(body);
```

```
//do not copy environment here since I will have to deep copy the environment in other places besides just here
```

```
//so do copying outside, then pass frozen copy into the closure
```

```
pv_clos->env = env;
```

```
pv_closure->pval_closure = pv_clos;
```

```
return pv_closure;
```

```
}
```

```
bool check_protected_symbols(char* name)
```

```
{
```

```
    const char *protected_syms[] = {"+", "-", "*", "/", "=", ">", "<", ">=", "<=", "!=", "quit", "head",  
    "tail", "cons", "not", "type", "ord", "chr", "input", "output", "cell", "!", ":", "str", "print", "error",  
    "eval", "read", "load"}; //, "get-file", "put-file";
```

```
    int length = sizeof(protected_syms) / sizeof(protected_syms[0]);
```

```
    for (int i = 0; i < length; i++)
```

```
    {
```

```
        if (strcmp(name, protected_syms[i]) == 0)
```

```
        {
```

```
            return true;
```

```
        }
```

```
    }
```

```
    return false;
```

```
}
```

```
void delete_env(environment* env)
{
    //base case, if env is NULL then return
    if (env == NULL) { return; }

    //delete local bindings
    binding *head = env->bindings;

    while (head != NULL)
    {
        binding *temp = head;
        char *temp_name = head->name;
        pval *temp_value = head->value;

        pval_delete(temp_value);
        free(temp_name);

        head = head->next;
        free(temp);
    }
    free(env);
}
```

```
void delete_env_chain(environment* env)
{

```

```

//base if the parent is NULL then return
if (env == NULL) { return; };

//grab env's parent
environment* temp_parent = env->parent;

//delete local bindings and the env
delete_env(env);

//walk the chain into the parent too
delete_env_chain(temp_parent);
}

environment* create_env()
{
    environment* env = (environment *)malloc(sizeof(environment));

    env->bindings = NULL;
    env->parent = NULL;

    return env;
}

environment* copy_env(environment *env)
{
    //base case: if parent is NULL then return new_env as NULL
    if (env == NULL) { return NULL; }

```

```
//create new env copy
```

```
environment *new_env = create_env();
```

```
//add the local bindings to env_env copy
```

```
binding* head = env->bindings;
```

```
while (head != NULL)
```

```
{
```

```
    env_add_binding(new_env, create_binding(head->name, pval_copy(head->value)));
```

```
    head = head->next;
```

```
}
```

```
//else: recursive copy env->parent all the way up until env->parent is NULL
```

```
//then return new_env that is copied
```

```
new_env->parent = copy_env(env->parent);
```

```
return new_env;
```

```
}
```

```
void add_env_parent(environment* env, environment* parent)
```

```
{
```

```
    if (env->parent == NULL)
```

```
    {
```

```
        env->parent = parent;
```

```
        return;
```

```
    }
```

```

    return;
}

binding* create_binding(char* name, pval* value)
{
    binding* new_binding = (binding *)malloc(sizeof(binding));

    new_binding->name = strdup(name);
    new_binding->value = value;
    new_binding->next = NULL;

    return new_binding;
}

```

```

void env_add_binding(environment* env, binding* func)
{
    //if no bindings present, add to the env as first binding
    if (env->bindings == NULL)
    {
        env->bindings = func;
        return;
    }
}

```

```

// override binding if it is found in the current env.
binding* temp = env->bindings;

```

```
while (temp != NULL)
{
    if (strcmp(temp->name, func->name) == 0)
    {
        pval* temp_val = temp->value;

        temp->value = func->value;

        pval_delete(temp_val);

        char* func_name = func->name;
        free(func_name);
        free(func);

        return;
    }
    temp = temp->next;
}
```

//add binding to the end if not found

```
temp = env->bindings;
while(temp->next != NULL)
{
    temp = temp->next;
}
temp->next = func;
```

```

    return;
}

pval* lookup_env_binding(environment* env, char* binding_name)
{
    //base case first which is search the bindings in current local env
    binding* temp = env->bindings;
    while (temp != NULL)
    {
        if (strcmp(temp->name, binding_name) == 0)
        {
            return pval_copy(temp->value);
        }
        temp = temp->next;
    }
    //recursive case search the parent env for the binding
    if (env->parent != NULL)
    {
        return lookup_env_binding(env->parent, binding_name);
    }

    //err_msg if bindings cannot be found in the parent or the local env
    char *err_msg = "error: unable to find binding for symbol...";
    return pval_error(pval_symbol(err_msg));
}

```

```

pval* pval_copy(pval* pv)
{
    if (pv->pval_type == PVAL_BOOL)
    {
        return pval_bool(pv->pval_bool);
    }
    else if (pv->pval_type == PVAL_NUMBER)
    {
        return pval_number(pv->pval_number);
    }
    else if (pv->pval_type == PVAL_SYMBOL)
    {
        return pval_symbol(pv->pval_symbol);
    }
    else if (pv->pval_type == PVAL_ERROR)
    {
        return pval_error(pval_copy(pv->pval_error));
    }
    else if (pv->pval_type == PVAL_STRING)
    {
        return pval_string(pv->pval_string->str, pv->pval_string->str_len);
    }
    else if (pv->pval_type == PVAL_FUNCTION)
    {
        pv->reference_count++;
    }
}

```

```

    return pv;
}
else if(pv->pval_type == PVAL_CLOSURE)
{
    pv->reference_count++;
    return pv;
}
else if (pv->pval_type == PVAL_CELL)
{
    pv->reference_count++;
    return pv;
}
else if(pv->pval_type == PVAL_LIST)
{
    pval* new_list = empty_list();

    list_child *temp = pv->pval_list->atoms_head;
    while (temp != NULL)
    {
        pval_add(new_list, pval_copy(temp->atom));
        temp = temp->next;
    }

    return new_list;
}
else

```

```

{
    char *err_msg = "error: Unable to copy current pval...";
    return pval_error(pval_symbol(err_msg));
}
};

```

```

pval* check_type(list_child* x)

```

```

{
    if (x->atom->pval_type == PVAL_BOOL)
    {
        char* err_msg = "$type error: got a bool when expecting a number...";
        return pval_error(pval_symbol(err_msg));
    }
    else if (x->atom->pval_type == PVAL_SYMBOL)
    {
        char* err_msg = "$type error: got a symbol when expecting a number...";
        return pval_error(pval_symbol(err_msg));
    }
    else if (x->atom->pval_type == PVAL_LIST)
    {
        char* err_msg = "$type error: got a list when expecting a number...";
        return pval_error(pval_symbol(err_msg));
    }
    else if (x->atom->pval_type == PVAL_FUNCTION)
    {
        char* err_msg = "$type error: got a function when expecting a number...";

```

```

        return pval_error(pval_symbol(err_msg));
    }
    else if(x->atom->pval_type == PVAL_STRING)
    {
        char* err_msg = "$type error: got a string when expecting a number...";
        return pval_error(pval_symbol(err_msg));
    }
    else
    {
        char* err_msg = "$type error: got a non-number when expecting a number...";
        return pval_error(pval_symbol(err_msg));
    }
}

```

```

pval* add(pval* list)
{
    list_child* temp = list->pval_list->atoms_head;

    int64_t sum = 0;

    while(temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_NUMBER)
        {
            sum = sum + temp->atom->pval_number;
        }
    }
}

```

```

    else
    {
        return check_type(temp);
    }
    temp = temp->next;
}

return pval_number(sum);
}

pval* subtract(pval* list)
{
    list_child* temp = list->pval_list->atoms_head;

    //checks for arity 0
    if (temp == NULL)
    {
        char* err_msg = "$error: requires more than 0 args...";
        return pval_error(pval_symbol(err_msg));
    }

    int64_t difference = 0;
    int64_t count = 0;

    while (temp != NULL)
    {

```

```

if (temp->atom->pval_type == PVAL_NUMBER)
{
    if (count == 0)
    {
        difference = temp->atom->pval_number;
    }
    else
    {
        difference = difference - temp->atom->pval_number;
    }
}
else
{
    return check_type(temp);
}
count++;
temp = temp->next;
}

```

//check for arity 1

//if count is 1, then while loop only ran 1 time.

```

if (count == 1)
{
    difference = 0 - difference;
}

```

```
    return pval_number(difference);  
};
```

```
pval* multiply(pval* list)  
{  
    list_child* temp = list->pval_list->atoms_head;  
  
    int64_t product = 1;  
  
    while (temp != NULL)  
    {  
        if (temp->atom->pval_type == PVAL_NUMBER)  
        {  
            product = product * temp->atom->pval_number;  
        }  
        else{  
            return check_type(temp);  
        }  
        temp = temp->next;  
    }  
  
    return pval_number(product);  
};
```

```
pval* divide(pval* list)  
{
```

```
list_child* temp = list->pval_list->atoms_head;
```

```
int64_t quotient = 0;
```

```
int64_t count = 0;
```

```
while (temp != NULL)
```

```
{
```

```
    if (temp->atom->pval_type == PVAL_NUMBER)
```

```
    {
```

```
        if (count == 0)
```

```
        {
```

```
            quotient = temp->atom->pval_number;
```

```
        }
```

```
    else
```

```
    {
```

```
        if (temp->atom->pval_number == 0)
```

```
        {
```

```
            char* err_msg = "error: cannot divide by 0!";
```

```
            return pval_error(pval_symbol(err_msg));
```

```
        }
```

```
    else
```

```
    {
```

```
        if ((quotient == INT64_MIN) && (temp->atom->pval_number == -1))
```

```
        {
```

```
            char* err_msg = "error: cannot divide by INT64_MIN";
```

```

        return pval_error(pval_symbol(err_msg));
    }

    if((quotient ^ temp->atom->pval_number) < 0 && (quotient %
temp->atom->pval_number != 0)) {
        quotient = quotient / temp->atom->pval_number;
        quotient--;
    }
    else
    {
        quotient = quotient / temp->atom->pval_number;

    }
}

}

}
else
{
    return check_type(temp);
}

count++;

temp = temp->next;

}

if (count < 2)
{
    char* err_msg = "error: division requires at least 2 arguments...";
    return pval_error(pval_symbol(err_msg));
}

```

```
}
```

```
return pval_number(quotient);
```

```
};
```

```
pval* equals(pval* list)
```

```
{
```

```
list_child* temp = list->pval_list->atoms_head;
```

```
bool b = true;
```

```
if (temp != NULL)
```

```
{
```

```
if (temp->atom->pval_type == PVAL_NUMBER)
```

```
{
```

```
int64_t reference = temp->atom->pval_number;
```

```
temp = temp->next;
```

```
while (temp != NULL)
```

```
{
```

```
if (temp->atom->pval_type == PVAL_NUMBER)
```

```
{
```

```
if (reference != temp->atom->pval_number)
```

```
{
```

```
b = false;
```

```
return pval_bool(b);
```

```
}
```

```

    }
    else
    {
        b = false;
        return pval_bool(b);
    }
    temp = temp->next;
}
}
else if (temp->atom->pval_type == PVAL_BOOL)
{
    bool reference = temp->atom->pval_bool;
    temp = temp->next;

    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_BOOL)
        {
            if (reference != temp->atom->pval_bool)
            {
                b = false;
                return pval_bool(b);
            }
        }
    }
    else
    {

```

```

        b = false;

        return pval_bool(b);
    }

    temp = temp->next;
}

}

else if (temp->atom->pval_type == PVAL_SYMBOL)
{
    char* reference = temp->atom->pval_symbol;

    temp = temp->next;

    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_SYMBOL)
        {
            if (strcmp(reference, temp->atom->pval_symbol) != 0)
            {
                b = false;

                return pval_bool(b);
            }
        }
        else
        {
            b = false;

            return pval_bool(b);
        }
    }
}

```

```

        temp = temp->next;
    }

}

else if (temp->atom->pval_type == PVAL_STRING)
{
    pv_str *reference = temp->atom->pval_string;
    temp = temp->next;

    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_STRING)
        {
            if (temp->atom->pval_string->str_len != reference->str_len)
            {
                b = false;
                return pval_bool(b);
            }
            if (memcmp(temp->atom->pval_string->str, reference->str, reference->str_len) != 0)
            {
                b = false;
                return pval_bool(b);
            }
        }
    }
    else
    {

```

```

        b = false;

        return pval_bool(b);
    }

    temp = temp->next;
}

}

else if(temp->atom->pval_type == PVAL_LIST)
{
    struct list *reference = temp->atom->pval_list;

    temp = temp->next;

    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_LIST)
        {
            //checks if list lengths are the same

            if (temp->atom->pval_list->list_count != reference->list_count)
            {
                b = false;

                return pval_bool(b);
            }

            list_child* cmp_list = temp->atom->pval_list->atoms_head;

            list_child* ref_list = reference->atoms_head;

            while (cmp_list != NULL)
            {

```

```
pval* temp_list = empty_list();  
pval_add(temp_list, pval_copy(ref_list->atom));  
pval_add(temp_list, pval_copy(cmp_list->atom));
```

```
pval* bl = equals(temp_list);  
if (bl->pval_bool == false)  
{  
    pval_delete(temp_list);  
    return bl;  
}  
pval_delete(bl);  
pval_delete(temp_list);  
cmp_list = cmp_list->next;  
ref_list = ref_list->next;  
}  
}  
else  
{  
    b = false;  
    return pval_bool(b);  
}  
temp = temp->next;  
}  
}  
else if (temp->atom->pval_type == PVAL_FUNCTION)
```

```

{
    pval* reference = temp->atom;
    temp = temp->next;

    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_FUNCTION)
        {
            if (temp->atom != reference)
            {
                b = false;
                return pval_bool(b);
            }
        }
        else
        {
            b = false;
            return pval_bool(b);
        }
        temp = temp->next;
    }
}

else if (temp->atom->pval_type == PVAL_CLOSURE)
{
    pval* reference = temp->atom;
    temp = temp->next;
}

```

```

while (temp != NULL)
{
    if (temp->atom->pval_type == PVAL_CLOSURE)
    {

        if (temp->atom != reference)
        {
            b = false;
            return pval_bool(b);
        }
    }
    else
    {
        b = false;
        return pval_bool(b);
    }
    temp = temp->next;
}

else if (temp->atom->pval_type == PVAL_CELL)
{
    pval* reference = temp->atom;
    temp = temp->next;
    while (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_CELL)

```

```
{

    if (temp->atom != reference)
    {
        b = false;
        return pval_bool(b);
    }
}
else
{
    b = false;
    return pval_bool(b);
}
temp = temp->next;
}
}
else
{
    b = false;
    return pval_bool(b);
}

}
return pval_bool(b);
};
```

```

pval* gt(pval* list)
{
    list_child *temp = list->pval_list->atoms_head;

    bool b = true;

    if (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_NUMBER)
        {
            int64_t reference = temp->atom->pval_number;
            temp = temp->next;

            while (temp != NULL)
            {
                if (temp->atom->pval_type == PVAL_NUMBER)
                {
                    if (reference <= temp->atom->pval_number)
                    {
                        b = false;
                        return pval_bool(b);
                    }
                }
                else
                {
                    reference = temp->atom->pval_number;
                }
            }
        }
    }
}

```

```

    }
    else
    {
        return check_type(temp);
    }
    temp = temp->next;
}
}
else{
    return check_type(temp);
}
}
return pval_bool(b);
};

```

```

pval* lt(pval* list)
{
    list_child *temp = list->pval_list->atoms_head;

    bool b = true;

    if (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_NUMBER)
        {
            int64_t reference = temp->atom->pval_number;

```

```

temp = temp->next;

while (temp != NULL)
{
    if (temp->atom->pval_type == PVAL_NUMBER)
    {
        if (reference >= temp->atom->pval_number)
        {
            b = false;
            return pval_bool(b);
        }
        else
        {
            reference = temp->atom->pval_number;
        }
    }
    else
    {
        return check_type(temp);
    }
    temp = temp->next;
}
}
else{
    return check_type(temp);
}

```

```

    }
    return pval_bool(b);
};

pval* gte(pval* list)
{
    list_child *temp = list->pval_list->atoms_head;

    bool b = true;

    if (temp != NULL)
    {
        if (temp->atom->pval_type == PVAL_NUMBER)
        {
            int64_t reference = temp->atom->pval_number;
            temp = temp->next;

            while (temp != NULL)
            {
                if (temp->atom->pval_type == PVAL_NUMBER)
                {
                    if (reference < temp->atom->pval_number)
                    {
                        b = false;
                        return pval_bool(b);
                    }
                }
            }
        }
    }
}

```

```

        else
        {
            reference = temp->atom->pval_number;
        }
    }
    else
    {
        return check_type(temp);
    }
    temp = temp->next;
}
}
else{
    return check_type(temp);
}
}
return pval_bool(b);
};

```

```

pval* lte(pval* list)

```

```

{
    list_child *temp = list->pval_list->atoms_head;

```

```

    bool b = true;

```

```

    if (temp != NULL)

```

```

{
    if (temp->atom->pval_type == PVAL_NUMBER)
    {
        int64_t reference = temp->atom->pval_number;
        temp = temp->next;

        while (temp != NULL)
        {
            if (temp->atom->pval_type == PVAL_NUMBER)
            {
                if (reference > temp->atom->pval_number)
                {
                    b = false;
                    return pval_bool(b);
                }
            }
            else
            {
                reference = temp->atom->pval_number;
            }
        }
        else
        {
            return check_type(temp);
        }
        temp = temp->next;
    }
}

```

```

    }
    else{
        return check_type(temp);
    }
}
return pval_bool(b);
};

```

```

pval* not_eq(pval* list)
{
    if (list->pval_list->atoms_head != NULL)
    {
        pval* b = equals(list);
        if (b->pval_bool == false)
        {
            pval_delete(b);
            return pval_bool(true);
        }
        pval_delete(b);
        return pval_bool(false);
    }
    return pval_bool(false);
};

```

```

pval* modulo(pval* list)
{

```

```

if (list->pval_list->list_count != 2)
{
    char *err_msg = "error: arity-error (modulo) must accept exactly 2 arguments.";
    return pval_error(pval_symbol(err_msg));
}

if (list->pval_list->atoms_head->atom->pval_type != PVAL_NUMBER)
{
    char *err_msg = "error: type-error (modulo) not a number.";
    return pval_error(pval_symbol(err_msg));
}

if (list->pval_list->atoms_head->next->atom->pval_type != PVAL_NUMBER)
{
    char *err_msg = "error: type-error (modulo) not a number.";
    return pval_error(pval_symbol(err_msg));
}

if (list->pval_list->atoms_head->next->atom->pval_number == 0)
{
    char *err_msg = "error: value-error (modulo) cannot mod by 0.";
    return pval_error(pval_symbol(err_msg));
}

if ((list->pval_list->atoms_head->atom->pval_number == INT64_MIN) &&
(list->pval_list->atoms_head->next->atom->pval_number == -1))
{

```

```

    char *err_msg = "error: value-error (modulo) cannot mod by INT64_MIN by -1.";
    return pval_error(pval_symbol(err_msg));
}

```

```

pval* quotient = divide(list);

int64_t remainder = list->pval_list->atoms_head->atom->pval_number-
quotient->pval_number*list->pval_list->atoms_head->next->atom->pval_number;

```

```

pval_delete(quotient);
return pval_number(remainder);
}

```

```

pval* head(pval* list)

```

```

{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (head) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->atom->pval_type == PVAL_LIST &&
list->pval_list->atoms_head->atom->pval_list->list_count == 0)
    {
        char *err_msg = "error: value-error (head) cannot have an empty list () as an argument.";
        return pval_error(pval_symbol(err_msg));
    };

    if (list->pval_list->atoms_head->atom->pval_type != PVAL_LIST) {
        char *err_msg = "error: type-error (head) can only accept non-empty lists as an argument.";
    }
}

```

```

        return pval_error(pval_symbol(err_msg));
    }
    return pval_copy(list->pval_list->atoms_head->atom->pval_list->atoms_head->atom);
};

pval* tail(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (tail) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->atom->pval_type == PVAL_LIST &&
list->pval_list->atoms_head->atom->pval_list->list_count == 0)
    {
        char *err_msg = "error: value-error (tail) cannot have an empty list () as an argument.";
        return pval_error(pval_symbol(err_msg));
    }
};

if (list->pval_list->atoms_head->atom->pval_type != PVAL_LIST) {
    char *err_msg = "error: type-error (tail) can only accept non-empty lists as an argument.";
    return pval_error(pval_symbol(err_msg));
}

pval* new_list = empty_list();

list_child* temp = list->pval_list->atoms_head->atom->pval_list->atoms_head->next;

while (temp != NULL)

```

```

{
    pval_add(new_list, pval_copy(temp->atom));
    temp = temp->next;
}

return new_list;
};

pval* cons(pval* list)
{
    if (list->pval_list->list_count != 2)
    {
        char *err_msg = "error: arity-error (cons) must accept exactly 2 arguments.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->next->atom->pval_type != PVAL_LIST)
    {
        char *err_msg = "error: type-error (cons) 2nd argument must be a list (empty or non-empty).";
        return pval_error(pval_symbol(err_msg));
    }

    pval* new_list = empty_list();
    pval_add(new_list, pval_copy(list->pval_list->atoms_head->atom));

    list_child* temp = list->pval_list->atoms_head->next->atom->pval_list->atoms_head;

```

```

while (temp != NULL)
{
    pval_add(new_list, pval_copy(temp->atom));
    temp = temp->next;
}
return new_list;
};

```

```

pval* not(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (not) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
}

```

```

bool b = false;
if (list->pval_list->atoms_head->atom->pval_type == PVAL_BOOL)
{
    if (list->pval_list->atoms_head->atom->pval_bool == false)
    {
        b = true;
        return pval_bool(b);
    }
}
}

```

```

    return pval_bool(b);
};

pval* type(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (type) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->atom->pval_type == PVAL_SYMBOL)
    {
        char *type = "symbol";
        return pval_symbol(type);
    }

    if (list->pval_list->atoms_head->atom->pval_type == PVAL_STRING)
    {
        char *type = "string";
        return pval_symbol(type);
    }

    else if (list->pval_list->atoms_head->atom->pval_type == PVAL_BOOL)
    {
        char *type = "bool";
        return pval_symbol(type);
    }
}

```

```

else if (list->pval_list->atoms_head->atom->pval_type == PVAL_NUMBER)
{
    char *type = "number";
    return pval_symbol(type);
}
else if (list->pval_list->atoms_head->atom->pval_type == PVAL_LIST)
{
    char *type = "list";
    return pval_symbol(type);
}
else if (list->pval_list->atoms_head->atom->pval_type == PVAL_FUNCTION)
{
    char *type = "function";
    return pval_symbol(type);
}
else if (list->pval_list->atoms_head->atom->pval_type == PVAL_CLOSURE)
{
    char* type = "function";
    return pval_symbol(type);
}
else if (list->pval_list->atoms_head->atom->pval_type == PVAL_CELL)
{
    char* type = "cell";
    return pval_symbol(type);
}
//add for string

```

```

else
{
    char *err_msg = "undefined type";
    return pval_error(pval_symbol(err_msg));
}
};

pval* ord(pval* list)
{
    // check number of arguments being accepted
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (ord) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    // check argument data types
    if (list->pval_list->atoms_head->atom->pval_type != PVAL_STRING)
    {
        char* err_msg = "$type error: (ord) didn't get a string when expecting a string...";
        return pval_error(pval_symbol(err_msg));
    }

    pval* new_list = empty_list();

    pv_str *pval_str = list->pval_list->atoms_head->atom->pval_string;

    for (int j=0; j<pval_str->str_len; j++)

```

```

{
    pval_add(new_list, pval_number((int64_t)(unsigned char)pval_str->str[j]));
}

return new_list;

};

pval* str_to_chr(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (chr) must accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    // check argument data types
    if (list->pval_list->atoms_head->atom->pval_type != PVAL_LIST)
    {
        char* err_msg = "$type error: (chr) didn't get a list when expecting a list...";
        return pval_error(pval_symbol(err_msg));
    }

    list_child* temp = list->pval_list->atoms_head->atom->pval_list->atoms_head;

    char* temp_str = (char
*)malloc((list->pval_list->atoms_head->atom->pval_list->list_count+1)*sizeof(char));

    int i = 0;

```

```

while (temp != NULL)
{
    if (temp->atom->pval_type != PVAL_NUMBER)
    {
        free(temp_str);

        char *err_msg = "error: (chr) didn't get a number when expecting a number.";

        return pval_error(pval_symbol(err_msg));
    }
    if (temp->atom->pval_number > 255)
    {
        free(temp_str);

        char *err_msg = "error: (chr) list element > 255";

        return pval_error(pval_symbol(err_msg));
    }
    if (temp->atom->pval_number < 0)
    {
        free(temp_str);

        char *err_msg = "error: (chr) list element < 0";

        return pval_error(pval_symbol(err_msg));
    }
    temp_str[i] = (unsigned char)temp->atom->pval_number;

    i++;

    temp = temp->next;
}

pval* pv_string = pval_string(temp_str, i);

```

```
    free(temp_str);  
    return pv_string;  
};
```

```
pval* input(pval *list)  
{  
    if (list->pval_list->list_count != 0)  
    {  
        char *err_msg = "error: arity-error (input) must accept exactly 0 arguments."  
        return pval_error(pval_symbol(err_msg));  
    }  
}
```

```
char *input = (char *)malloc(4097*sizeof(char));
```

```
if (fgets(input, 4097, stdin) == NULL)  
{  
    pval* pv_string = pval_string(input, 0);  
    free(input);  
    return pv_string;  
}
```

```
int input_len = strlen(input);
```

```
if (input_len > 0 && input[input_len-1] == '\n')  
{  
    input[input_len-1] = '\0';  
    input_len--;
```

```
}
```

```
if (input_len > 0 && input[input_len-1] == '\r') { input[input_len-1] = '\0'; }
```

```
pval* pv_string = pval_string(input, input_len);
```

```
free(input);
```

```
return pv_string;
```

```
};
```

```
pval* output(pval* list)
```

```
{
```

```
list_child *temp = list->pval_list->atoms_head;
```

```
while (temp != NULL)
```

```
{
```

```
if (temp->atom->pval_type != PVAL_STRING)
```

```
{
```

```
char *err_msg = "error: type-error (output) only accepts strings as arguments.";
```

```
return pval_error(pval_symbol(err_msg));
```

```
}
```

```
temp = temp->next;
```

```
}
```

```
temp = list->pval_list->atoms_head;
```

```
while (temp != NULL)
```

```

{
    for (int j = 0; j < temp->atom->pval_string->str_len; j++)
    {
        printf("%c", temp->atom->pval_string->str[j]);
    }
    temp = temp->next;
}
return pval_bool(true);
};

```

```

pval* create_cell(pval* list)

```

```

{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (cell) can only accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->atom->pval_type == PVAL_ERROR)
    {
        char *err_msg = "error: type-error (cell) cannot accept an error as an argument.";
        return pval_error(pval_symbol(err_msg));
    }

    return pval_cell(pval_copy(list->pval_list->atoms_head->atom));
};

```

```

pval* deref_cell(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: arity-error (!) can only accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
    if (list->pval_list->atoms_head->atom->pval_type != PVAL_CELL)
    {
        char *err_msg = "error: type-error (!) can only accept cell as argument.";
        return pval_error(pval_symbol(err_msg));
    }

    return pval_copy(list->pval_list->atoms_head->atom->pval_cell);
};

```

```

pval* write_cell(pval* list)
{
    if (list->pval_list->list_count != 2)
    {
        char *err_msg = "error: arity-error (:=) can only accept exactly 2 arguments.";
        return pval_error(pval_symbol(err_msg));
    }

    if (list->pval_list->atoms_head->atom->pval_type != PVAL_CELL)

```

```

{
    char *err_msg = "error: type-error (:=) can only accept cell in argument 1.";
    return pval_error(pval_symbol(err_msg));
}

```

```

pval* temp = list->pval_list->atoms_head->atom->pval_cell;

```

```

list->pval_list->atoms_head->atom->pval_cell =
pval_copy(list->pval_list->atoms_head->next->atom);

```

```

pval_delete(temp);
return pval_copy(list->pval_list->atoms_head->atom->pval_cell);
};

```

```

pval* str(pval* list)

```

```

{
    if (list->pval_list->list_count != 1)
    {
        char *err_msg = "error: type-error (str) can only accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
    pval* pv = list->pval_list->atoms_head->atom;
    if (pv->pval_type == PVAL_BOOL)
    {
        if (pv->pval_bool) return pval_string("#t", 2);
        return pval_string("#f", 2);
    }
}

```

```

else if (pv->pval_type == PVAL_NUMBER)
{
    int length = snprintf(NULL, 0, "%" PRId64, pv->pval_number);
    char* temp_str = (char *) malloc((length+1)*sizeof(char));

    snprintf(temp_str, length+1, "%" PRId64, pv->pval_number);

    pval* pv_str = pval_string(temp_str, length);
    free(temp_str);
    return pv_str;
}
else if (pv->pval_type == PVAL_SYMBOL)
{
    int str_len = strlen(pv->pval_symbol);
    return pval_string(pv->pval_symbol, str_len);
}
else if (pv->pval_type == PVAL_FUNCTION)
{
    int len = snprintf(NULL, 0, "$builtin{%s}", pv->pval_func->func_name);
    char *temp_str = (char *)malloc((len+1)*sizeof(char));

    snprintf(temp_str, len+1, "$builtin{%s}", pv->pval_func->func_name);

    pval* pv_str = pval_string(temp_str, len);
    free(temp_str);
}

```

```

    return pv_str;
}
else if (pv->pval_type == PVAL_CLOSURE)
{
    pval* holding_list = empty_list();

    int len = 2;
    if (pv->pval_closure->is_macro == true)
    {
        pval_add(holding_list, pval_string("$macro{", 7));
        len+=7;
    }
    else {
        pval_add(holding_list, pval_string("$lambda{", 8));
        len+=8;
    }

    pval* temp_list = empty_list();
    pval_add(temp_list, pval_copy(pv->pval_closure->body));

    pval* temp_arglist = empty_list();
    pval_add(temp_arglist, pval_copy(pv->pval_closure->arglist));

    pval* arglist_str = str(temp_arglist);
    pval* body_str = str(temp_list);

```

```

pval_add(holding_list, arglist_str);
pval_add(holding_list, pval_string(" ", 1));
pval_add(holding_list, body_str);
pval_add(holding_list, pval_string("}", 1));

len+=arglist_str->pval_string->str_len;
len+=body_str->pval_string->str_len;

if (pv->pval_closure->fn_name != NULL)
{
    int sym_len = strlen(pv->pval_closure->fn_name->pval_symbol);
    pval_add(holding_list, pval_string("@", 1));
    pval_add(holding_list, pval_string(pv->pval_closure->fn_name->pval_symbol,
sym_len));
    len++; // for @
    len += sym_len;
}
char* temp_str = (char *)malloc(len*sizeof(char));
int idx = 0;
list_child* temp = holding_list->pval_list->atoms_head;
while (temp != NULL)
{
    memcpy(temp_str+idx, temp->atom->pval_string->str,
temp->atom->pval_string->str_len);
    idx += temp->atom->pval_string->str_len;
    temp = temp->next;
}

```

```

    pval* pv_str = pval_string(temp_str, idx);
    pval_delete(holding_list);
    pval_delete(temp_list);
    pval_delete(temp_arglist);

    free(temp_str);
    return pv_str;
}
else if (pv->pval_type == PVAL_CELL)
{

    pval* temp_list = empty_list();
    pval_add(temp_list, pval_copy(pv->pval_cell));

    pval* temp = str(temp_list);
    int temp_len = temp->pval_string->str_len;
    int pt_len = snprintf(NULL, 0, "%p", (void *)temp);

    char* temp_str = (char *)malloc((temp_len+pt_len+8+1)*sizeof(char));
    snprintf(temp_str, (temp_len+pt_len+1+8), "$cell{%s}@%p", temp->pval_string->str, (void
*)pv);

    pval* pv_str = pval_string(temp_str, temp_len+pt_len+8);
    pval_delete(temp_list);
    pval_delete(temp);
    free(temp_str);
    return pv_str;
}

```

```

}
else if (pv->pval_type == PVAL_LIST)
{
    list_child *temp = pv->pval_list->atoms_head;

    pval* holding_list = empty_list();

    int len = 3; // for ( ) /0
    while (temp != NULL)
    {
        pval* temp_list = empty_list();
        pval_add(temp_list, pval_copy(temp->atom));
        pval* temp_str = str(temp_list);

        len = len + temp_str->pval_string->str_len + 1;
        pval_add(holding_list, pval_copy(temp_str));
        pval_delete(temp_str);
        pval_delete(temp_list);

        temp = temp->next;
    }
    len--;

    char *temp_str = (char *)malloc(len*sizeof(char));
    int idx = 0;
    temp_str[idx++] = '(';

```

```

temp = holding_list->pval_list->atoms_head;

while (temp != NULL)
{
    if (idx != 1) temp_str[idx++] = ' ';

    memcpy(temp_str+ idx, temp->atom->pval_string->str,
temp->atom->pval_string->str_len);

    idx += temp->atom->pval_string->str_len;

    temp = temp->next;
}

temp_str[idx++] = ')';

pv* pv_str = pval_string(temp_str, idx);

pval_delete(holding_list);

free(temp_str);

return pv_str;
}

else if (pv->pval_type == PVAL_STRING)
{
    char *temp_str = (char *)malloc((4*pv->pval_string->str_len+3)*sizeof(char));

    int count = 0;

    temp_str[count++] = '\"';

    for (int j=0; j<pv->pval_string->str_len; j++)
    {
        if(pv->pval_string->str[j] == '\n')
        {
            temp_str[count++] = '\\';

            temp_str[count++] = 'n';

```

```

}
else if(pv->pval_string->str[j] == '\\')
{
    temp_str[count++] = '\\';
    temp_str[count++] = '\\';
}
else if(pv->pval_string->str[j] == '\0')
{
    temp_str[count++] = '\\';
    temp_str[count++] = 'x';
    temp_str[count++] = '0';
    temp_str[count++] = '0';
}
else if(pv->pval_string->str[j] == '\"')
{
    temp_str[count++] = '\\';
    temp_str[count++] = '\"';
}
else if ((pv->pval_string->str[j] >= 32) && (pv->pval_string->str[j] <= 126))
{
    temp_str[count++] = pv->pval_string->str[j];
}
else
{
    snprintf(temp_str+count, 5, "\\x%02x", (unsigned char)pv->pval_string->str[j]);
    count+=4;
}

```

```

    }
}
temp_str[count++] = '\\';

pval* pv_str = pval_string(temp_str, count);
free(temp_str);
return pv_str;
}
else
{
    pval* temp_list = empty_list();
    pval_add(temp_list, pval_copy(pv->pval_error));

    pval* temp = str(temp_list);
    int temp_len = temp->pval_string->str_len;

    char* temp_str = (char *)malloc((temp_len+8+1)*sizeof(char));
    snprintf(temp_str, (temp_len+1+8), "$error{%s}", temp->pval_string->str);

    pval* pv_str = pval_string(temp_str, temp_len+8);
    pval_delete(temp_list);
    pval_delete(temp);
    free(temp_str);
    return pv_str;
}
};

```

```

pval* print(pval* list)
{
    pval* output_list = empty_list();
    list_child *temp = list->pval_list->atoms_head;
    while (temp != NULL)
    {
        pval* tmp_list = empty_list();
        pval_add(tmp_list, pval_copy(temp->atom));
        pval_add(output_list, str(tmp_list));
        if (temp->next != NULL)
        {
            pval_add(output_list, pval_string(" ", 1));
        }
        pval_delete(tmp_list);
        temp = temp->next;
    }
    pval_add(output_list, pval_string("\n", 1));
    pval* otpt = output(output_list);
    pval_delete(output_list);
    return otpt;
};

```

```

pval* make_list(pval* list)
{
    return pval_copy(list);
}

```

```

}

pval* error(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char* err_msg = "error: arity-error (error) can only accept 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    return pval_error(pval_copy(list->pval_list->atoms_head->atom));
};

Node* parse_help(pval* list)
{
    pval* pv_str = list->pval_list->atoms_head->atom;
    // create NULL terminated copy

    char* temp_str = (char *)malloc((pv_str->pval_string->str_len+1)*sizeof(char));

    memcpy(temp_str, pv_str->pval_string->str, pv_str->pval_string->str_len);
    temp_str[pv_str->pval_string->str_len] = '\0';

    Node* token_list = NULL;
    token_list = lex(token_list, temp_str);

    Node* cursor = token_list;
    free(temp_str);

```

```

    return cursor;
}

pval* read(pval* list)
{
    if (list->pval_list->list_count != 1)
    {
        char* err_msg = "error: arity-error (read) can only accept 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
    if (list->pval_list->atoms_head->atom->pval_type != PVAL_STRING)
    {
        char* err_msg = "error: type-error (read) can only accept strings as an argument.";
        return pval_error(pval_symbol(err_msg));
    }

    Node* cursor = parse_help(list);

    if (cursor == NULL) {
        char* err_msg = "error: value-error (read) empty string found.";
        return pval_error(pval_symbol(err_msg));
    }
    Node* head = cursor;

    pval* parse_exp = parse_expression(&cursor);

```

```
if (cursor != NULL)
{
    char *err_msg = "error: (read) value-error.";
    pval *temp = parse_exp;
    parse_exp = pval_error(pval_symbol(err_msg));
    pval_delete(temp);
}

deleteList(head);

return parse_exp;
};
```

```
pval* create_list(pval* list)
{
    return pval_copy(list);
};
```

```
// pval* get_file(pval* list)
// {};
```

```
// pval* put_file(pval* list)
// {};
```

```
pval* exit_prg(pval* list)
```

```

{
    list_child* temp = list->pval_list->atoms_head;

    if (temp != NULL)
    {
        char* err_msg = "$error: arity-error, quit cannot have > 0 arguments...";
        return pval_error(pval_symbol(err_msg));
    }

    terminate = true;
    return pval_number(0);
};

```

```

pval* eval_tree(environment* env, pval* x)
{
    if (x->pval_type == PVAL_BOOL || x->pval_type == PVAL_NUMBER || x->pval_type ==
PVAL_ERROR || x->pval_type == PVAL_FUNCTION || x->pval_type == PVAL_STRING)
    {
        return pval_copy(x);
    }

    else if (x->pval_type == PVAL_SYMBOL)
    {
        return lookup_env_binding(env, x->pval_symbol);
    }

    else if (x->pval_type == PVAL_LIST)
    {
        if (x->pval_list->atoms_head == NULL)

```

```

{
    return pval_copy(x);
}
else
{
    //add def (special form) here

    //checks if the first element in the list is a symbol
    if (x->pval_list->atoms_head->atom->pval_type == PVAL_SYMBOL) {
        //checks if the symbol is "def"
        if (strcmp(x->pval_list->atoms_head->atom->pval_symbol, "def") == 0)
        {
            if (x->pval_list->list_count == 3)
            {
                if (x->pval_list->atoms_head->next->atom->pval_type != PVAL_SYMBOL)
                {
                    char* err_msg = "$error: type-error (def) 2nd argument must be a symbol...";
                    return pval_error(pval_symbol(err_msg));
                }
                if
(check_protected_symbols(x->pval_list->atoms_head->next->atom->pval_symbol))
                {
                    printf("%s\n", x->pval_list->atoms_head->next->atom->pval_symbol);
                    char* err_msg = "$error: type-error (def) 2nd argument cannot be a protected
symbol...";
                    return pval_error(pval_symbol(err_msg));
                }
            }
        }
    }
}

```

```

    pval *eval_def = eval_tree(env, x->pval_list->atoms_head->next->next->atom);

    if (eval_def->pval_type == PVAL_ERROR)
    {
        return eval_def;
    }

    env_add_binding(env,
create_binding(x->pval_list->atoms_head->next->atom->pval_symbol, eval_def));

    return pval_copy(eval_def);
}

else if (x->pval_list->list_count == 4)
{
    if (x->pval_list->atoms_head->next->atom->pval_type != PVAL_SYMBOL)
    {
        char* err_msg = "$error: type-error (def) 2nd argument must be a symbol...";
        return pval_error(pval_symbol(err_msg));
    }

    if
(check_protected_symbols(x->pval_list->atoms_head->next->atom->pval_symbol))
    {
        char* err_msg = "$error: type-error (def) 2nd argument cannot be a protected
symbol...";

        return pval_error(pval_symbol(err_msg));
    }

    pval* def_list = empty_list();

    char *fn = "fn";

    pval_add(def_list, pval_symbol(fn));

```

```

list_child *temp = x->pval_list->atoms_head->next;
while (temp != NULL)
{
    pval_add(def_list, pval_copy(temp->atom));
    temp = temp->next;
}
pval *eval_def_lst = eval_tree(env, def_list);
pval_delete(def_list);
if (eval_def_lst->pval_type != PVAL_ERROR)
{
    env_add_binding(env,
create_binding(x->pval_list->atoms_head->next->atom->pval_symbol, eval_def_lst));
}
return pval_copy(eval_def_lst);

}
else
{
    char *err_msg = "error: arity-error (def) must have exactly 2 or 3 arguments.";
    return pval_error(pval_symbol(err_msg));
}
}
else if (strcmp(x->pval_list->atoms_head->atom->pval_symbol, "if") == 0)
{
    if (x->pval_list->list_count == 3 || x->pval_list->list_count == 4)
    {
        //evaluate 1st subform of if statement

```

```

pval* eval_first = eval_tree(env, x->pval_list->atoms_head->next->atom);

if (eval_first->pval_type == PVAL_ERROR) { return eval_first; };


if (eval_first->pval_type == PVAL_BOOL)
{
    if (eval_first->pval_bool == false)
    {
        if (x->pval_list->list_count == 3)
        {
            pval_delete(eval_first);

            return pval_bool(false);

        }

        //eval 3rd subform if there is a 3rd subform and 1st subform is false

        pval* eval_third = eval_tree(env,
x->pval_list->atoms_head->next->next->next->atom);

        pval_delete(eval_first);

        return eval_third;

    }

}

//eval 2nd subform if 1st subform is true

pval *eval_second = eval_tree(env,
x->pval_list->atoms_head->next->next->atom);

pval_delete(eval_first);

return eval_second;

}

else

{

```

```

char *err_msg = "error: arity-error (if) must have minimum 2 args and maximum
3 args.";

return pval_error(pval_symbol(err_msg));

}

}

else if (strcmp(x->pval_list->atoms_head->atom->pval_symbol, "quote") == 0)
{
    if (x->pval_list->list_count == 2) {
        return pval_copy(x->pval_list->atoms_head->next->atom);
    }
    else
    {
        char *err_msg = "error: arity-error (quote) must have 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
}

else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "fn") == 0)
{
    if ((x->pval_list->list_count != 3) && (x->pval_list->list_count != 4))
    {
        char *err_msg = "error: arity-error (fn) must have minimum 2 args and maximum
3 args.";

        return pval_error(pval_symbol(err_msg));
    }

}

//anonymous user defined functions

//fn is first element

```

```

if (x->pval_list->list_count == 3)
{
    // check that arglist is a list
    pval* arglist = x->pval_list->atoms_head->next->atom;
    if (arglist->pval_type != PVAL_LIST)
    {
        char *err_msg = "error: arglist-error (fn) arglist () be a list.";
        return pval_error(pval_symbol(err_msg));
    }
    if (arglist->pval_list->atoms_head != NULL)
    {
        list_child *temp = arglist->pval_list->atoms_head;

        //check each element in arg list that it is symbol and that & is second to last
        int i = 0;
        while (temp != NULL)
        {
            if (temp->atom->pval_type != PVAL_SYMBOL)
            {
                char *err_msg = "error: arglist-error (fn) arglist can only contain
symbols.";
                return pval_error(pval_symbol(err_msg));
            }
            if ((strcmp(temp->atom->pval_symbol, "&") == 0) && (i !=
arglist->pval_list->list_count-2))
            {
                char *err_msg = "error: arglist-error (fn) & symbol msut be only in 2nd to
last position in arglist.";

```

```

        return pval_error(pval_symbol(err_msg));
    }

    i++;

    temp = temp->next;
}

//check for duplicate symbols in arglist
list_child* outer = arglist->pval_list->atoms_head;

while (outer->next != NULL)
{
    list_child *inner = outer->next;

    while (inner != NULL)
    {
        if (strcmp(outer->atom->pval_symbol, inner->atom->pval_symbol) == 0)
        {
            char *err_msg = "error: arglist-error (fn) arglist cannot contain duplicate
symbols.";

            return pval_error(pval_symbol(err_msg));
        }

        inner = inner->next;
    }

    outer = outer->next;
}

return pval_closure(false, NULL, arglist,
x->pval_list->atoms_head->next->next->atom, copy_env(env));

```

```

}

//named user defined functions
if (x->pval_list->list_count == 4)
{
    if (x->pval_list->atoms_head->next->atom->pval_type != PVAL_SYMBOL)
    {
        char *err_msg = "error: type-error (fn) fn name mst be a symbol.";
        return pval_error(pval_symbol(err_msg));
    }

    // check that arglist is a list
    pval* arglist = x->pval_list->atoms_head->next->next->atom;
    if (arglist->pval_type != PVAL_LIST)
    {
        char *err_msg = "error: arglist-error (fn) arglist () be a list.";
        return pval_error(pval_symbol(err_msg));
    }

    if (arglist->pval_list->atoms_head != NULL)
    {
        list_child *temp = arglist->pval_list->atoms_head;

        //check each element in arg list that it is symbol and that & is second to last
        int i = 0;
        while (temp != NULL)
        {
            if (temp->atom->pval_type != PVAL_SYMBOL)
            {

```

```

        char *err_msg = "error: arglist-error (fn) arglist can only contain
symbols.";

        return pval_error(pval_symbol(err_msg));
    }

    if ((strcmp(temp->atom->pval_symbol, "&") == 0) && (i !=
arglist->pval_list->list_count-2))
    {
        char *err_msg = "error: arglist-error (fn) & symbol msut be only in 2nd to
last position in arglist.";

        return pval_error(pval_symbol(err_msg));
    }

    i++;

    temp = temp->next;
}

//check for duplicate symbols in arglist
list_child* outer = arglist->pval_list->atoms_head;

while (outer->next != NULL)
{
    list_child *inner = outer->next;

    while (inner != NULL)
    {
        if (strcmp(outer->atom->pval_symbol, inner->atom->pval_symbol) == 0)
        {

            char *err_msg = "error: arglist-error (fn) arglist cannot contain duplicate
symbols.";

```

```

        return pval_error(pval_symbol(err_msg));
    }

    inner = inner->next;

}

outer = outer->next;

}

}

return pval_closure(false, x->pval_list->atoms_head->next->atom, arglist,
x->pval_list->atoms_head->next->next->next->atom, copy_env(env));
}

}

else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "do") == 0)
{
    if (x->pval_list->list_count == 1)
    {
        bool b = true;

        return pval_bool(b);
    }

    list_child* temp = x->pval_list->atoms_head->next;

    // pval* temp_list = empty_list();

    while (temp != NULL)
    {
        pval* subform = eval_tree(env, temp->atom);

        // pval_add(temp_list, pval_copy(subform));
    }
}

```

```

    if (subform->pval_type == PVAL_ERROR)
    {
        // pval_delete(temp_list);

        return subform;
    }
    if (temp->next == NULL)
    {
        // pval_delete(temp_list);

        temp = temp->next;

        return subform;
    }
    pval_delete(subform);
    temp = temp->next;
}
}
else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "loop") == 0)
{
    if(x->pval_list->list_count != 2)
    {
        char *err_msg = "error: arity-error (loop) can only accept exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }
    while (true)
    {
        pval* subform = eval_tree(env, x->pval_list->atoms_head->next->atom);

```

```

        //break out of loop if error

        if (subform->pval_type == PVAL_ERROR)
        {
            return subform;
        }

        if (subform->pval_type == PVAL_BOOL && subform->pval_bool == false)
        {
            pval_delete(subform);
            return pval_bool(true);
        }

        pval_delete(subform);
    }

}

else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "try") == 0)
{
    if (x->pval_list->list_count != 2)
    {
        char* err_msg = "arity-error (try) can only accept 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    pval* subform = eval_tree(env, x->pval_list->atoms_head->next->atom);

```

```
pval* return_list = empty_list();
```

```
if (subform->pval_type == PVAL_ERROR)
```

```
{
```

```
    pval_add(return_list, pval_bool(false));
```

```
    pval_add(return_list, pval_copy(subform->pval_error));
```

```
    pval_delete(subform);
```

```
    return return_list;
```

```
}
```

```
pval_add(return_list, pval_bool(true));
```

```
pval_add(return_list, subform);
```

```
return return_list;
```

```
}
```

```
else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "and") == 0)
```

```
{
```

```
    if(x->pval_list->list_count == 1)
```

```
    {
```

```
        return pval_bool(true);
```

```
    }
```

```
list_child* temp = x->pval_list->atoms_head->next;
```

```
while (temp != NULL)
```

```
{
```

```
    pval* subform = eval_tree(env, temp->atom);
```

```

        if (subform->pval_type == PVAL_ERROR)
        {
            return subform;
        }

        if (subform->pval_type == PVAL_BOOL && subform->pval_bool == false)
        {
            return subform;
        }
        if (temp->next == NULL)
        {
            return subform;
        }

        pval_delete(subform);
        temp = temp->next;
    }
}

else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "or") == 0)
{
    if(x->pval_list->list_count == 1)
    {
        return pval_bool(false);
    }

    list_child* temp = x->pval_list->atoms_head->next;

```

```

while (temp != NULL)
{
    pval* subform = eval_tree(env, temp->atom);

    if (subform->pval_type == PVAL_ERROR)
    {
        return subform;
    }

    if (subform->pval_type == PVAL_BOOL && subform->pval_bool != false)
    {
        return subform;
    }

    if (subform->pval_type != PVAL_BOOL && subform->pval_type !=
PVAL_ERROR)
    {
        return subform;
    }

    pval_delete(subform);
    temp = temp->next;
}

return pval_bool(false);
}

else if(strcmp(x->pval_list->atoms_head->atom->pval_symbol, "let") == 0)
{
    if (x->pval_list->list_count == 1)

```

```

    {
        char* err_msg = "arity-error: (let) accepts a minimum of 1 argument which is the
body.";

        return pval_error(pval_symbol(err_msg));
    }
    if ((x->pval_list->list_count-1) % 2 == 0)
    {
        char* err_msg = "arity-error: (let) can only accept odd number of arguments.";
        return pval_error(pval_symbol(err_msg));
    }

    // form is (let body)
    //base case (let body)
    if (x->pval_list->list_count == 2)
    {
        pval* body = eval_tree(env, x->pval_list->atoms_head->next->atom);
        return body;
    }

    //form is (let name1 expr1 ... nameN exprN body)
    //recursive cases
    if (x->pval_list->atoms_head->next->atom->pval_type != PVAL_SYMBOL)
    {
        char* err_msg = "type-error: (let) 1st argument must be a symbol.";
        return pval_error(pval_symbol(err_msg));
    }

```

```
//make -> (fn)
```

```
pval* fn_list = empty_list();
```

```
char* fn = "fn";
```

```
pval_add(fn_list, pval_symbol(fn));
```

```
//make -> (name)
```

```
pval* name_list = empty_list();
```

```
pval* name = x->pval_list->atoms_head->next->atom;
```

```
pval_add(name_list, pval_copy(name));
```

```
//make -> (fn (name))
```

```
pval_add(fn_list, name_list);
```

```
//make -> (let rest...)
```

```
pval* inner_let = empty_list();
```

```
char* let = "let";
```

```
pval_add(inner_let, pval_symbol(let));
```

```
list_child* temp = x->pval_list->atoms_head->next->next->next;
```

```
while (temp != NULL)
```

```
{
```

```
    pval_add(inner_let, pval_copy(temp->atom));
```

```
    temp = temp->next;
```

```
}
```

```
//make -> (fn (name) (let rest))
```

```

    pval_add(fn_list, inner_let);

    //pull expr
    pval* expr = x->pval_list->atoms_head->next->next->atom;

    // make -> ()
    pval* outer = empty_list();

    //make -> ((fn (name) (let rest)) expr)
    pval_add(outer, fn_list);
    pval_add(outer, pval_copy(expr));

    pval* result = eval_tree(env, outer);
    pval_delete(outer);
    return result;
}

else if (strcmp(x->pval_list->atoms_head->atom->pval_symbol, "eval") == 0)
{
    if (x->pval_list->list_count != 2)
    {
        char* err_msg = "arity-error: (eval) accepts a exactly 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    pval* first_eval = eval_tree(env, x->pval_list->atoms_head->next->atom);
    pval* snd_eval = eval_tree(env, first_eval);

```

```

    pval_delete(first_eval);

    return snd_eval;

}

else if (strcmp(x->pval_list->atoms_head->atom->pval_symbol, "load") == 0)
{
    if (x->pval_list->list_count != 2)
    {
        char* err_msg = "error: arity-error (load) can only accept 1 argument.";
        return pval_error(pval_symbol(err_msg));
    }

    pval* eval_filename = eval_tree(env, x->pval_list->atoms_head->next->atom);
    if (eval_filename->pval_type != PVAL_STRING)
    {
        pval_delete(eval_filename);

        char* err_msg = "error: type-error (load) can only accept strings as an
argument.";

        return pval_error(pval_symbol(err_msg));
    }

    FILE *fptr;

    fptr = fopen(eval_filename->pval_string->str, "r");

    pval_delete(eval_filename);

    if (fptr == NULL)
    {

```

```
    char* err_msg = strerror(errno);/"error: file-error (load) error loading file.";
    return pval_error(pval_symbol(err_msg));
}
```

```
fseek(fp_ptr, 0L, SEEK_END);
int64_t fsize = ftell(fp_ptr);
```

```
if (fsize < 0)
{
    fclose(fp_ptr);
    char* err_msg = "error: file-error (load) file size < 0 bytes.";
    return pval_error(pval_symbol(err_msg));
}
if (fsize == 0)
{
    fclose(fp_ptr);
    return pval_number(0);
}
```

```
rewind(fp_ptr);
```

```
char* buffer = (char *)malloc((fsize)*sizeof(char));
```

```
int64_t n = fread(buffer, 1, fsize, fp_ptr);
fclose(fp_ptr);
```

```
pval* buffer_str = pval_string(buffer, n);
```

```
pval* buffer_list = empty_list();
```

```
pval_add(buffer_list, buffer_str);
```

```
Node* parsed_buffer = parse_help(buffer_list);
```

```
Node* temp = parsed_buffer;
```

```
int count = 0;
```

```
while (temp != NULL && terminate == false)
```

```
{
```

```
    pval* parsed_exp = parse_expression(&temp);
```

```
    pval* eval = eval_tree(env, parsed_exp);
```

```
    if (eval->pval_type == PVAL_ERROR)
```

```
    {
```

```
        pval_delete(parsed_exp);
```

```
        free(buffer);
```

```
        deleteList(parsed_buffer);
```

```
        pval_delete(buffer_list);
```

```
        return eval;
```

```
    }
```

```
    count++;
```

```
    pval_delete(parsed_exp);
```

```

        pval_delete(eval);
    }

    free(buffer);
    deleteList(parsed_buffer);
    pval_delete(buffer_list);

    return pval_number(count);
}
}

pval *evaluate = eval_tree(env, x->pval_list->atoms_head->atom);
// built in functions here
if (evaluate->pval_type == PVAL_FUNCTION)
{
    pval* temp_list = empty_list();

    list_child* temp = x->pval_list->atoms_head->next;

    while(temp != NULL)
    {
        pval* result = eval_tree(env, temp->atom);
        if (result->pval_type == PVAL_ERROR) {

            pval_delete(evaluate);
            pval_delete(temp_list);

```

```

        return result;
    }

    pval_add(temp_list, result);

    temp = temp->next;
}

pval* temp_val = evaluate->pval_func->pval_fn_def(temp_list);

pval_delete(evaluate);
pval_delete(temp_list);

return temp_val;
}

//user defined functions here

//head has already been evaluted

//taking closure path here
else if (evaluate->pval_type == PVAL_CLOSURE)
{
    // evaluate the arguments here left to right, propagating left most error

    pval* temp_list = empty_list();

    list_child* temp = x->pval_list->atoms_head->next;

    while(temp != NULL)
    {
        pval* result = eval_tree(env, temp->atom);

```

```

if (result->pval_type == PVAL_ERROR) {

    pval_delete(evaluate);
    pval_delete(temp_list);
    return result;
}

pval_add(temp_list, result);
temp = temp->next;
}

//arity check here

list_child* check_args = evaluate->pval_closure->arglist->pval_list->atoms_head;
bool has_ampersand = false;

while (check_args != NULL)
{
    if (strcmp(check_args->atom->pval_symbol, "&") == 0)
    {
        has_ampersand = true;
        check_args = NULL;
        break;
    }
    check_args = check_args->next;
}

if (has_ampersand == false)

```

```

    {
        if (evaluate->pval_closure->arglist->pval_list->list_count !=
temp_list->pval_list->list_count)
        {
            pval_delete(temp_list);

            pval_delete(evaluate);

            char *err_msg = "error: arity-error (evaluating fn) got different number of
arguments than expected.";

            return pval_error(pval_symbol(err_msg));
        }
    }

    if (temp_list->pval_list->list_count <
evaluate->pval_closure->arglist->pval_list->list_count - 2)
    {
        pval_delete(temp_list);

        pval_delete(evaluate);

        char *err_msg = "error: arity-error (evaluating fn) got < 2 arguments when expecting
at least 2.";

        return pval_error(pval_symbol(err_msg));
    }

    //create a new environment (or frame)

    //the parent environment is the closure's frozen env

    environment* new_frame = create_env();

    add_env_parent(new_frame, evaluate->pval_closure->env);

```

```

//bind the parameters of the function here

temp = temp_list->pval_list->atoms_head;

list_child* temp_arg = evaluate->pval_closure->arglist->pval_list->atoms_head;
while (temp_arg != NULL)
{
    if (strcmp(temp_arg->atom->pval_symbol, "&") == 0)
    {
        temp_arg = temp_arg->next;
        pval* new_list = empty_list();

        while (temp != NULL)
        {
            pval_add(new_list, pval_copy(temp->atom));
            temp = temp->next;
        }

        env_add_binding(new_frame, create_binding(temp_arg->atom->pval_symbol,
pval_copy(new_list)));
        pval_delete(new_list);
        temp_arg = temp_arg->next;
    }
    else
    {
        env_add_binding(new_frame, create_binding(temp_arg->atom->pval_symbol,
pval_copy(temp->atom)));
        temp_arg = temp_arg->next;
        temp = temp->next;
    }
}

```

```
    }  
}
```

//if the function is named, then bind the function closure (as a pval) to the newly created environment

```
    if (evaluate->pval_closure->fn_name != NULL)  
    {  
        env_add_binding(new_frame,  
create_binding(evaluate->pval_closure->fn_name->pval_symbol, pval_copy(evaluate)));  
    }
```

//evaluate the function body in this new frame

```
pval* result = eval_tree(new_frame, evaluate->pval_closure->body);
```

//delete the frame after body is evaluated

```
delete_env(new_frame);
```

```
pval_delete(evaluate);
```

```
pval_delete(temp_list);
```

```
return result;
```

```
}
```

```
else if (evaluate->pval_type == PVAL_ERROR)
```

```
{
```

```
    return evaluate;
```

```
}
```

```
pval_delete(evaluate);
```

```
char *err_msg = "error: inapplicable head";
```

```

        return pval_error(pval_symbol(err_msg));
    }
}

else {
    char *err_msg = "error: cannot evalute tree in current form...";
    return pval_error(pval_symbol(err_msg));
}
};

```

Globals.c

```

#include <stdbool.h>

#include "../include/globals.h"

bool terminate = false;

```

Lexer.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
#include <ctype.h>

#include "../include/lexer.h"

```

```
char *createToken(char command) {  
  
    char *token = malloc(sizeof(char)*2);  
    token[0] = command;  
    token[1] = '\\0';  
  
    return token;  
}
```

```
Node *createNode(char *token) {  
  
    Node *newNode = (Node *)malloc(sizeof(Node));  
  
    newNode->token = token;  
    newNode->next = NULL;  
  
    return newNode;  
}
```

```
void append(Node** head, char *token) {
```

```
Node *new_node = createNode(token);

Node *last = *head;

if (*head == NULL) {

    *head = new_node;
    return;
}

while (last->next != NULL) {
    last = last->next;
}

last->next = new_node;
return;
}

void printList(Node *node)
{
    while (node != NULL)
    {
```

```
    printf(" %s ", node->token);

    node = node->next;
}
}

void deletelist(Node *head)
{
    if (head == NULL) {
        return;
    }

    while (head != NULL) {
        Node *temp = head;
        char *temp_str = temp->token;

        free(temp_str);

        head = head->next;
        free(temp);
    }
}

Node *lex(Node *head, char* command) {
    int command_len = strlen(command);
```

```

if (command_len > 0 && command[command_len-1] == '\n')
{
    command[command_len-1] = '\0';
    command_len--;
}

if (command_len > 0 && command[command_len-1] == '\r') { command[command_len-1] = '\0'; }

for (int i=0; command[i] != '\0'; i++) {

    char c = command[i];
    char next_c = command[i+1];

    //tokenize ( ) ' _
    if (c == '(' || c == ')' || c == '\\' || c == '_' || c == '%' || c == '&')
    {
        append(&head, createToken(c));
    }
    else if (c == ' ' || c == '\r' || c == '\n')
    {
        continue;
    }
}

```

```

//tokenize >=, <=, !=, and :=
else if ((c == '>' || c == '<' || c == '!' || c == ':') && next_c == '=')
{
    char *temp = (char *)calloc(3, sizeof(char));

    temp[0] = command[i];
    temp[1] = command[i+1];
    temp[2] = '\\0';

    append(&head, temp);

    i++;
}

//tokenize -0(-9)
else if ((c == '+' && (next_c >= '0' && next_c <= '9')) || (c == '-' &&
(next_c >= '0' && next_c <= '9')))
{
    char *temp = (char *)calloc(4097, sizeof(char));

    int temp_idx = 1;

    temp[0] = command[i];

    for (int j = i+1; (command[j] != ')') && (command[j] != '\\0') &&
(command[j] != ' ') && (command[j] != '\\")); j++)
    {

```

```

        if (command[j] >= '0' && command[j] <= '9')
        {
            temp[temp_idx] = command[j];
            temp_idx++;
        }
        else{
            break;
        }
        i = j;
    }
    append(&head, temp);
}

//tokenize * - / + = < > !

    else if (c == '*' || c == '/' || c == '+' || c == '-' || c == '=' || c ==
'>' || c == '<' || c == '!')
    {
        append(&head, createToken(c));
    }
    else if (c == '#' && (next_c == 'f' || next_c == 't'))
    {
        char *temp = (char *)calloc(3, sizeof(char));

        temp[0] = command[i];
        temp[1] = command[i+1];
        temp[2] = '\0';
    }

```

```
append(&head, temp);

    i++;
}
//tokenize 0-9
else if (c >= '0' && c <= '9')
{
    char *temp = (char *)calloc(4097, sizeof(char));
    int temp_idx = 0;

    for (int j = i; (command[j] != ')') && (command[j] != '\\0') &&
(command[j] != ' ') && (command[j] != "\\"); j++)
    {
        if (command[j] >= '0' && command[j] <= '9') {
            temp[temp_idx] = command[j];
            temp_idx++;
        }
        else
        {
            break;
        }
        i = j;
    }

    append(&head, temp);
```

```

}

else if (c == ';')
{

    for (int j = i; (command[j] != '\0') && command[j] != '\n'; j++)
    {
        i = j;
    }

}

else if (c == '\"')
{
    //gets string length
    int len = 0;
    for (int j = i+1; (command[j] != '\0') && (command[j] != '\"'); j++)
    {
        if (command[j] == '\\' && command[j+1] != '\0')
        {
            j++;
            len++;
        }
        len++;
    }

    //allocate token string
    char *temp = (char *)malloc((len+3)*sizeof(char));

```

```
int temp_idx = 1;

temp[0] = command[i];

for (int j = i+1; (command[j] != '\0'); j++)
{
    //checks for escape characters like \n \0 \\ \" \xZw
    if (command[j] == '\\' && command[j+1] != '\0')
    {
        temp[temp_idx] = command[j];
        temp_idx++;
        temp[temp_idx] = command[j+1];
        temp_idx++;
        j++;
        continue;
    }
    else{
        temp[temp_idx] = command[j];
        temp_idx++;
    }

    if (command[j] == '\"') { break; }
}

i = i+len+1;
```

```

        if (command[i] == '\0') { i--;}

        temp[temp_idx] = '\0';

        append(&head, temp);

    }

    //parse symbols

    else if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'))
    {

        char *temp = (char *)calloc(4097, sizeof(char));

        int temp_idx = 0;

        for (int j = i; (command[j] != ')') && (command[j] != '\0') &&
(command[j] != ' ') && (command[j] != '\"')); j++)
        {

            if(strchr("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ01
23456789+-*/%=<>!:.", command[j]) == NULL)

            {

                break;

            }

            temp[temp_idx] = command[j];

            temp_idx++;

        }

        i = j;

    }

```

```

        append(&head, temp);
    }
    else if (isspace((unsigned char)c) == 0)
    {
        printf("Invalid token error");
    }
}

return head;
}

```

Parser.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
#include <ctype.h>

#include "../include/parser.h"
#include "../include/eval.h"

//Parser function def
//pval constructor functions

```

```
pval* pval_func(pval_function pval_fn, char* func_name)
{
    pv_func *fn_struct = (pv_func *)malloc(sizeof(pv_func));

    fn_struct->func_name = strdup(func_name);
    fn_struct->pval_fn_def = pval_fn;

    pval* fn = (pval *)malloc(sizeof(pval));

    fn->reference_count = 1;
    fn->pval_type = PVAL_FUNCTION;
    fn->pval_func = fn_struct;

    return fn;
}
```

```
pval* pval_cell(pval* x)
{
    pval* pv_cell = (pval *)malloc(sizeof(pval));
```

```
pv_cell->reference_count = 1;

pv_cell->pval_type = PVAL_CELL;
pv_cell->pval_cell = x;


return pv_cell;
}

pval* pval_number(int64_t n)
{
    pval *pval_num = (pval *)malloc(sizeof(pval));

    pval_num->reference_count = 1;
    pval_num->pval_type = PVAL_NUMBER;
    pval_num->pval_number = n;

    return pval_num;
};

pval* pval_bool(bool b)
{
    pval* pval_b1 = (pval *)malloc(sizeof(pval));

    pval_b1->reference_count = 1;
    pval_b1->pval_type = PVAL_BOOL;
```

```
pval_bl->pval_bool = b;

return pval_bl;
};

pval* pval_symbol(char *symbol)
{
    pval* pval_sym = (pval *)malloc(sizeof(pval));

    pval_sym->reference_count = 1;
    pval_sym->pval_type = PVAL_SYMBOL;

    //duplicate the symbol tokens in the tokenized list so the token list can be
    freed after it is parsed.

    //need to free this symbol when defining pval_delete().

    pval_sym->pval_symbol = strdup(symbol);

    return pval_sym;
};

pval* pval_string(char *str, int str_len)
{
    pval* pval_str = (pval *)malloc(sizeof(pval));
```

```

    pval_str->reference_count = 1;
    pval_str->pval_type = PVAL_STRING;

    pv_str* pv_string = (pv_str *)malloc(sizeof(pv_str));
    pv_string->str_len = str_len;
    pv_string->str = (char *)malloc((str_len+1)*sizeof(char));
    memcpy((pv_string->str), str, str_len);
    pv_string->str[str_len] = '\0';

    pval_str->pval_string = pv_string;

    return pval_str;
}

pval* pval_error(pval* x)
{
    pval *error = (pval *)malloc(sizeof(pval));

    error->reference_count = 1;
    error->pval_type = PVAL_ERROR;
    error->pval_error = x;

```

```

        return error;
};

//makes empty list to add pvals to later by calling pval_add().
pval* empty_list()
{
    pval *em_list = (pval *)malloc(sizeof(pval));

    em_list->reference_count = 1;
    em_list->pval_type = PVAL_LIST;

    list *new_list = (list *)malloc(sizeof(list));
    new_list->list_count = 0;
    new_list->atoms_head = NULL;

    em_list->pval_list = new_list;

    return em_list;
};

//adds a pval element to the list passed into it.
void pval_add(pval* list, pval* elem)

```

```
{  
  
    list_child *new_child = (list_child *)malloc(sizeof(list_child));  
    new_child->atom = elem;  
    new_child->next = NULL;  
  
    if (list->pval_list->atoms_head == NULL) {  
  
        list->pval_list->atoms_head = new_child;  
        list->pval_list->list_count++;  
        return;  
    }  
  
    list_child *temp = list->pval_list->atoms_head;  
  
    while (temp->next != NULL) {  
        temp = temp->next;  
    }  
  
    temp->next = new_child;  
    list->pval_list->list_count++;  
    return;  
};
```

```
void pval_print(pval* pv)
{
    if (pv->pval_type == PVAL_BOOL)
    {
        if (pv->pval_bool == true)
        {
            printf("#t");
        }
        else
        {
            printf("#f");
        }
    }
    else if (pv->pval_type == PVAL_NUMBER)
    {
        printf("%lld", (long long)pv->pval_number);
    }
    else if (pv->pval_type == PVAL_SYMBOL)
    {
        printf("%s", pv->pval_symbol);
    }
    else if (pv->pval_type == PVAL_STRING)
    {
        printf("%c", '\\');
        for (int j=0; j<pv->pval_string->str_len; j++)
        {
            if(pv->pval_string->str[j] == '\\n')
```

```

    {
        printf("%c%c", '\\', 'n');
    }
    else if(pv->pval_string->str[j] == '\\')
    {
        printf("%c%c", '\\', '\\');
    }
    else if(pv->pval_string->str[j] == '\\0')
    {
        printf("%c%c%c%c", '\\', 'x', '0', '0');
    }
    else if(pv->pval_string->str[j] == '\"')
    {
        printf("%c%c", '\\', '\"');
    }
    else if ((pv->pval_string->str[j] >= 32) && (pv->pval_string->str[j]
<= 126))
    {
        printf("%c", pv->pval_string->str[j]);
    }
    else
    {
        printf("%c%c%02x", '\\', 'x', (unsigned
char)pv->pval_string->str[j]);
    }
}

printf("%c", '\\');

```

```

    }

    else if (pv->pval_type == PVAL_ERROR)
    {
        printf("$error{");
        pval_print(pv->pval_error);
        printf("}");
    }

    else if (pv->pval_type == PVAL_FUNCTION)
    {
        printf("$builtin{%s}", pv->pval_func->func_name);
    }

    else if (pv->pval_type == PVAL_LIST)
    {
        // case for printing out quote to ' if quote is a symbol

        if (pv->pval_list->atoms_head != NULL &&
pv->pval_list->atoms_head->atom->pval_type == PVAL_SYMBOL)
        {
            if (((pv->pval_list->atoms_head->next != NULL)) &&
(strncmp(pv->pval_list->atoms_head->atom->pval_symbol, "quote") == 0))
            {
                printf("'");
                pval_print(pv->pval_list->atoms_head->next->atom);
                return;
            }
        }

        printf("(");

        list_child *temp = pv->pval_list->atoms_head;

```

```
int i = 0;

while (temp != NULL)
{
    if (i != 0) { printf(" "); }

    pval_print(temp->atom);

    i++;

    temp = temp->next;
}

printf("\n");
}

else if (pv->pval_type == PVAL_CLOSURE)
{
    if (pv->pval_closure->is_macro == true)
    {
        printf("$marco{");
    }

    else {
        printf("$lambda{");
    }

    pval_print(pv->pval_closure->arglist);

    printf(" ");

    pval_print(pv->pval_closure->body);

    printf("}");

    if (pv->pval_closure->fn_name != NULL)
    {
        printf("@");

        pval_print(pv->pval_closure->fn_name);
    }
}
```

```

    }

}

else if (pv->pval_type == PVAL_CELL)
{
    printf("$cell{");
    pval_print(pv->pval_cell);
    printf("}%p", (void *)pv);
}
};

void pval_delete(pval* pv)
{
    if (pv == NULL)
    {
        return;
    }

    //decrement the reference count, if == 0, then delete the pval, else return.
    pv->reference_count--;

    if (pv->reference_count > 0)
    {
        return;
    }

    if (pv->pval_type == PVAL_BOOL)
    {
        free(pv);
    }
}

```

```
}

else if (pv->pval_type == PVAL_NUMBER)
{
    free(pv);
}

else if (pv->pval_type == PVAL_SYMBOL)
{
    char* temp_symbol = pv->pval_symbol;
    free(temp_symbol);

    free(pv);
}

else if (pv->pval_type == PVAL_ERROR)
{
    pval_delete(pv->pval_error);

    free(pv);

}

else if (pv->pval_type == PVAL_CELL)
{
    pval_delete(pv->pval_cell);

    free(pv);
```

```
}  
  
else if (pv->pval_type == PVAL_FUNCTION)  
{  
  
    pv_func* temp_fn = pv->pval_func;  
    char* temp_name = pv->pval_func->func_name;  
  
    free(temp_name);  
    free(temp_fn);  
    free(pv);  
}  
  
else if (pv->pval_type == PVAL_LIST)  
{  
    list_child *head = pv->pval_list->atoms_head;  
  
    while (head != NULL)  
    {  
        list_child *temp = head;  
        pval_delete(temp->atom);  
        head = head->next;  
  
        free(temp);  
    }  
  
    free(pv->pval_list);  
}
```

```
    free(pv);
}
else if (pv->pval_type == PVAL_STRING)
{
    pv_str *temp_pv_str = pv->pval_string;
    char *temp_str = temp_pv_str->str;
    free(temp_str);
    free(temp_pv_str);

    free(pv);
}
else if (pv->pval_type == PVAL_CLOSURE)
{
    pval* temp_fn = pv->pval_closure->fn_name;
    pval* temp_args = pv->pval_closure->arglist;
    pval* temp_body = pv->pval_closure->body;

    environment *temp_env = pv->pval_closure->env;

    closure *temp_closure = pv->pval_closure;

    pval_delete(temp_fn);
    pval_delete(temp_args);
```

```

    pval_delete(temp_body);

    delete_env_chain(temp_env);

    free(temp_closure);
    free(pv);
}
};

//grammar rule expression -> atom | list
pval* parse_expression(Node** current)
{
    if (strcmp((*current)->token, "(") == 0)
    {
        //consume the token -> (
        (*current) = (*current)->next;
        //call parse list?
        return parse_list(current);
    }
    else if (strcmp((*current)->token, ")") == 0)
    {
        char* error_message = "Syntax error, not expecting a )...";
        (*current) = (*current)->next;
        return pval_error(pval_symbol(error_message));
    }
}

```

```

else if (strcmp((*current)->token, "'") == 0)
{
    //this handles the short hand ' for quote outliend in phase 2

    pval* quote_list = empty_list();

    char* quote = "quote";
    pval_add(quote_list, pval_symbol(quote));

    if((*current)->next == NULL)
    {
        pval_delete(quote_list);
        (*current) = (*current)->next;
        char *err_msg = "error: ' incomplete parse.";
        return pval_error(pval_symbol(err_msg));
    }

    (*current) = (*current)->next;
    pval_add(quote_list, parse_expression(current));

    return quote_list;
}
else
{
    return parse_atom(current);
}

```

```

};

//grammar rule list -> '(' expression* ')' - * means 0 or more expressions in
between ()

pval* parse_list(Node** current)
{
    //create a new empty list to add atoms/other lists to

    pval* new_list = empty_list();

    //(*current) != NULL comes first, because if it's NULL, then the token will
also be NULL.

    while (((*current) != NULL) && (strcmp((*current)->token, ")") != 0))
    {
        pval_add(new_list, parse_expression(current));
    }

    if (((*current) != NULL) && strcmp((*current)->token, ")") == 0)
    {
        (*current) = (*current)->next;
    }

    return new_list;
};

```

```

//grammar rule atom -> bool | symbol | number

pval* parse_atom(Node** current)
{
    //parse booleans #t or #f

    // atom -> bool

    if (strcmp((*current)->token, "#t\0") == 0 || strcmp((*current)->token,
"#f\0") == 0)
    {
        bool temp = false;

        if (strcmp((*current)->token, "#t") == 0) {
            temp = true;
        }

        else if (strcmp((*current)->token, "#f") == 0)
        {
            temp = false;
        }

        pval* b = pval_bool(temp);

        (*current) = (*current)->next;

        return b;
    }

    //parse a number

    // atom -> number

    else if ((((*current)->token[0] == '-' || (*current)->token[0] == '+') &&
((*current)->token[1] >= 48 && (*current)->token[1] <= 57)) ||
((*current)->token[0] >= 48 && (*current)->token[0] <= 57)) //parse a number

```

```

{
    int64_t temp;

    temp = strtoll((*current)->token, NULL, 10);

    pval* number = pval_number(temp);

    (*current) = (*current)->next;
    return number;
}

//parse strings
// atom -> string
else if ((*current)->token[0] == '\\')
{
    char *temp_str = (char *)malloc(strlen((*current)->token)*sizeof(char));
    int pval_len = 0;
    int num_par = 0;
    for (int j=0; ((*current)->token[j] != '\\0'); j++)
    {
        // edge case #1 : check for \xZW (hexadecimal strings) 0-255
        if( ((*current)->token[j] == '\\') &&
            ((*current)->token[j+1] == 'x') &&
            (isxdigit((*current)->token[j+2])) &&
            (isxdigit((*current)->token[j+3]))
        )
    }
}

```

```

{
    unsigned int temp;

    sscanf(&((*current)->token[j+2]), "%2x", &temp);

    temp_str[pval_len] = (unsigned char)temp;
    pval_len++;
    j+=3;
}

//edge case #2: check for other escaped characters \n \0 \" '\\
else if (((*current)->token[j] == '\\') && ((*current)->token[j+1] ==
'n')
)
{
    temp_str[pval_len] = '\\n';
    pval_len++;
    j++;
}

else if (((*current)->token[j] == '\\') && ((*current)->token[j+1] ==
'0')
)
{
    temp_str[pval_len] = '\\0';
    pval_len++;
    j++;
}

else if (((*current)->token[j] == '\\') && ((*current)->token[j+1] ==
'\"'))

```

```

    )
    {
        temp_str[pval_len] = '\"';
        pval_len++;
        j++;
    }
    else if ((*current)->token[j] == '\\') && ((*current)->token[j+1] ==
'\\')
    )
    {
        temp_str[pval_len] = '\\';
        pval_len++;
        j++;
    }
    else if ((*current)->token[j] == '\"')
    {
        num_par++;
    }
    //get the rest of the characters
    else
    {
        temp_str[pval_len] = (*current)->token[j];
        pval_len++;
    }
}

```

```
if (num_par != 2)
{
    char* err_msg = "error: incomplete parse";
    free(temp_str);
    (*current) = (*current)->next;
    return pval_error(pval_symbol(err_msg));
}
```

```
(*current) = (*current)->next;
pval* pv_str = pval_string(temp_str, pval_len);
free(temp_str);
return pv_str;
```

```
}
//parse symbols
// atom -> symbol
else
{
    pval* symbol = pval_symbol(((current)->token));

    (*current) = (*current)->next;
    return symbol;
}
```

```
};
```

Psi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

#include "../include/lexer.h"
#include "../include/parser.h"
#include "../include/eval.h"
#include "../include/globals.h"

int main(int argc, char * argv[]) {
    //create global environment
    environment *global_env = create_env();

    //create and bindings to global env
    //phase 1 built ins
    env_add_binding(global_env, create_binding("+", pval_func(add, "+")));
    env_add_binding(global_env, create_binding("-", pval_func(subtract, "-")));
    env_add_binding(global_env, create_binding("*", pval_func(multiply, "*")));
    env_add_binding(global_env, create_binding("/", pval_func(divide, "/"));
```

```
env_add_binding(global_env, create_binding("quit", pval_func(exit_prg,
"quit"))));

env_add_binding(global_env, create_binding("=", pval_func(equals, "=")));

//phase 2 builtins

env_add_binding(global_env, create_binding("head", pval_func(head, "head")));
env_add_binding(global_env, create_binding("tail", pval_func(tail, "tail")));
env_add_binding(global_env, create_binding("type", pval_func(type, "type")));

env_add_binding(global_env, create_binding(">", pval_func(gt, ">")));
env_add_binding(global_env, create_binding("<", pval_func(lt, "<")));
env_add_binding(global_env, create_binding(">=", pval_func(gte, ">=")));
env_add_binding(global_env, create_binding("<=", pval_func(lte, "<=")));
env_add_binding(global_env, create_binding("!= ", pval_func(not_eq, "!=")));
env_add_binding(global_env, create_binding("not", pval_func(not, "not")));
env_add_binding(global_env, create_binding("_", empty_list()));
env_add_binding(global_env, create_binding("cons", pval_func(cons, "cons")));

env_add_binding(global_env, create_binding("ord", pval_func(ord, "ord")));
env_add_binding(global_env, create_binding("chr", pval_func(str_to_chr,
"chr")));

env_add_binding(global_env, create_binding("output", pval_func(output,
"output")));

env_add_binding(global_env, create_binding("input", pval_func(input,
"input")));
```

```

//phase 3 bindings

env_add_binding(global_env, create_binding("%", pval_func(modulo, "%")));
env_add_binding(global_env, create_binding("cell", pval_func(create_cell,
"cell")));

env_add_binding(global_env, create_binding("!", pval_func(deref_cell, "!")));
env_add_binding(global_env, create_binding(":= ", pval_func(write_cell,
":=")));

env_add_binding(global_env, create_binding("print", pval_func(print,
"print")));

env_add_binding(global_env, create_binding("str", pval_func(str, "str")));
env_add_binding(global_env, create_binding("list", pval_func(make_list,
"list")));

env_add_binding(global_env, create_binding("error", pval_func(error,
"error")));

env_add_binding(global_env, create_binding("read", pval_func(read, "read")));

int flag = 0;
while (terminate == false) {

    if (argc > 1)
    {
        pval* filename = pval_string(argv[1], strlen(argv[1]));

        pval* lst = empty_list();
        pval_add(lst, pval_symbol("load"));
        pval_add(lst, filename);

        pval* result = eval_tree(global_env, lst);

```

```
    if (result->pval_type == PVAL_ERROR)
    {
        pval_delete(lst);
        pval_print(result);
        pval_delete(result);
        terminate = true;
        flag = 1;
        break;
    }
    pval_delete(lst);
    pval_delete(result);
    terminate = true;
}
else
{
    char *command = (char *)calloc(4097, sizeof(char));
    printf("psi>");

    if (fgets(command, 4097, stdin) == NULL) {
        printf("No meaningful input found. Exiting program...");
        free(command);
        break;
    }
    else
```

```

{
    Node *token_list = NULL;
    token_list = lex(token_list, command);

    Node* head = token_list;

    Node* cursor = token_list;

    // printList(token_list);
    if (token_list == NULL) {
        free(command);
        continue;
    }
    pval* tree = parse_expression(&cursor);
    deleteList(head);

    //checks if the entire line was parsed
    if ((cursor != NULL ) && (tree->pval_type != PVAL_ERROR))
    {
        char *err_msg = "error: incomplete parse  cursor not NULL.";
        pval *temp = tree;
        tree = pval_error(pval_symbol(err_msg));
        pval_delete(temp);
    }
}

```

```

        pval* evaled_tree = eval_tree(global_env, tree);

        // pval_print(tree);
        pval_print(evaled_tree);
        printf("\n");

        if (evaled_tree->pval_type != PVAL_ERROR)
        {
            env_add_binding(global_env, create_binding("_",
pval_copy(evaled_tree)));
        }

        pval_delete(tree);
        pval_delete(evaled_tree);

        free(command);
    }

}

}

delete_env(global_env);

return flag;
}

```